

Control Structures II (Repetition)

Engr.Tehseen Ahsan

EC-110: Computing Fundamentals (CF)

Assistant Professor, Computer Engineering Department

HITEC University Taxila Cantt, Pakistan

Repetition

- The programs we have *studied previously* (in control structures-I) were based on *sequential statements* or *selection statements*.
- *In most programs, we need to repeat lines of code. Repetition allows the program to iterate a section of code multiple times.*
- *C++ provides three constructs for repeating a set of statements* without physically repeating them in the code: *while statements*, *for statements*, *do-while statements*.

Repetition Statements

- Three repetition statements are used in C++; the *while statement*, the *for statement*, and the *do-while statement*.

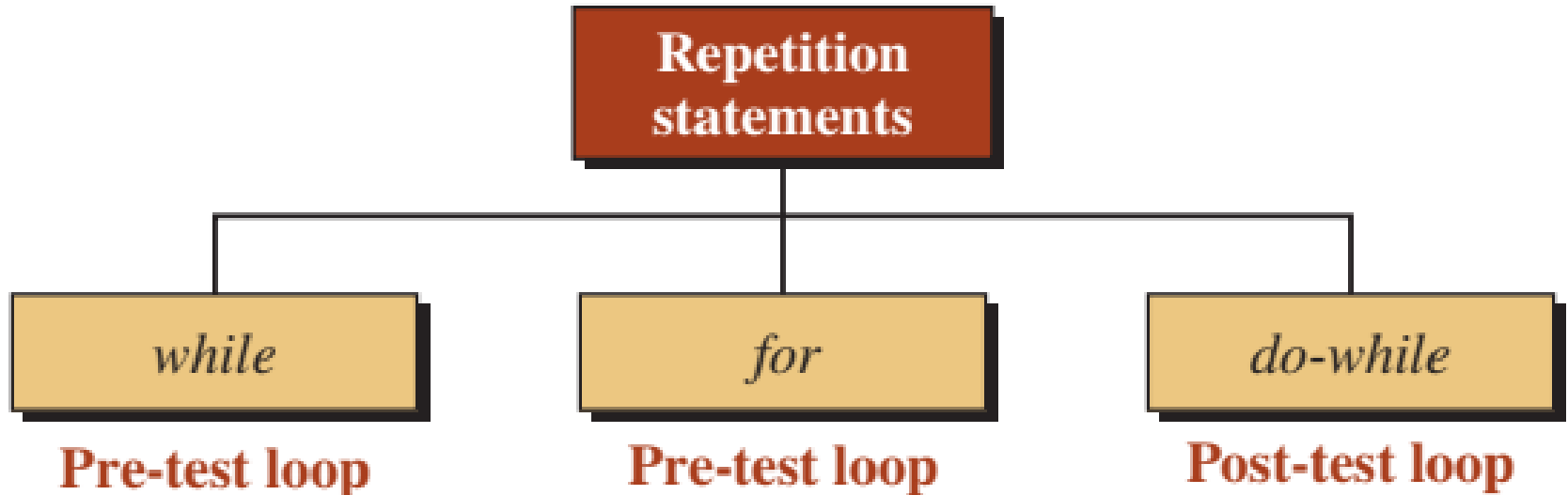


Figure 5

The *while* Statement / *while* Loop

- Figure 6 shows the flow diagram and the syntax of the *while statement*. The while statement is made of the reserved word while, followed by a Boolean expression (called the condition) in parenthesis, followed by a single statement, which is the body of the while statement.

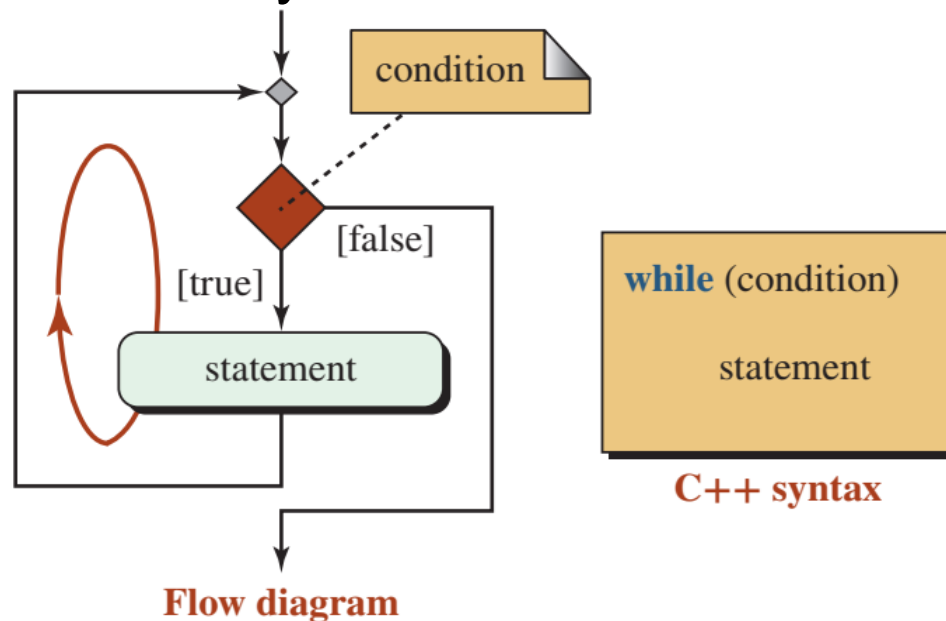


Figure 6

The *while* Statement / *while* Loop (*continued...*)

- The compound statement can be implemented using while loop.

```
while (condition)
{
    statement-1;
    ...
    statement-n;
}
```

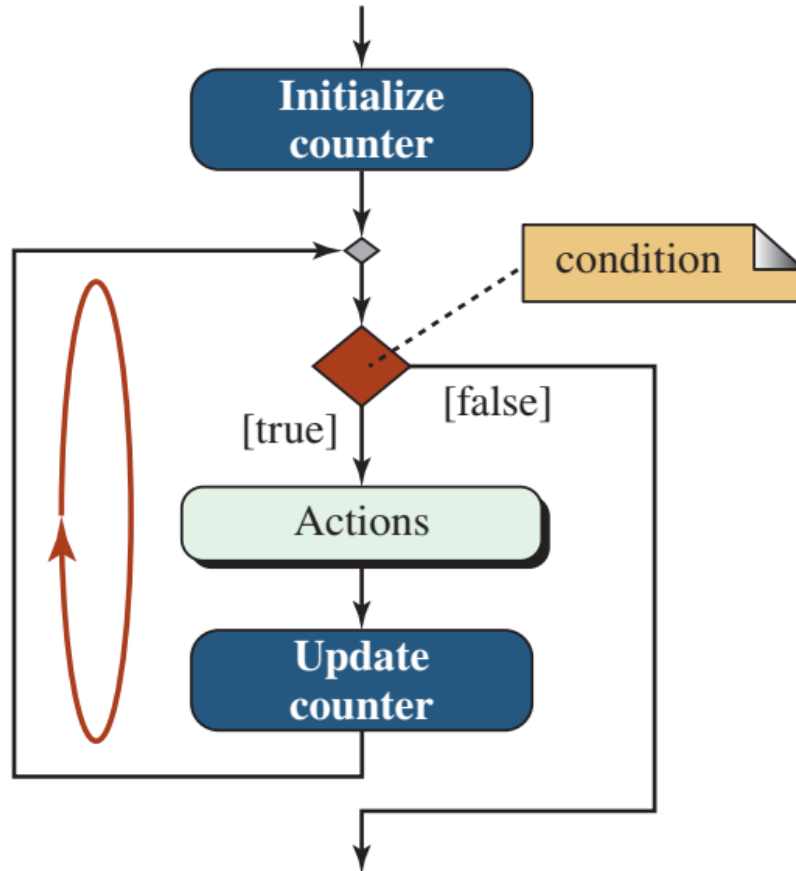
The *while* Statement / *while* Loop (*continued...*)

C++ provides two types of while statements: *counter-controlled* and *event-controlled*.

1. **Counter-Controlled *while* Statement**

Often, we know how many times the body of the loop must be repeated. In these cases, we can use a counter. *We can set the counter to an initial value before the loop,* increment or decrement the counter in each iteration, and stop the loop when we are done. For example, if we have decided to do 20 push-ups every morning, we are using a **counter-controlled loop**. We know how many times we want to repeat it. We count the number of push-ups and we stop when we are done. Figure 7 next slide shows a counter-controlled loop.

The *while* Statement / *while* Loop (continued...)



Flow diagram

```
Initialize counter
while (condition)
{
    Actions
    Update counter
}
```

Code

Figure 7: Flow diagram and the code for a counter-controlled while statement

The *while* Statement / *while* Loop (continued...)

A counter-controlled *while* statement

```
1  /*****
2  * Use of a while statement to print a message 10 times *
3  *****/
4  #include <iostream>
5  using namespace std;
6
7  int main ( )
8  {
9      // Declaration and initialization of counter
10     int counter = 0;
11     // While statement
12     while (counter < 10) // The number of repetition is fixed to 10
13     {
14         cout << "Hello world!" << endl;
15         counter++;
16     }
17     return 0;
18 }
```


The *while* Statement / *while* Loop (*continued...*)

Run:

```
Hello world!  
Hello world!  
Hello world!  
Hello world!  
Hello world!  
Hello world!  
Hello world!  
Hello world!  
Hello world!  
Hello world!  
Hello world!
```

Home Assignment 2

Program 1: (Use of while loop/counter-controlled loop)

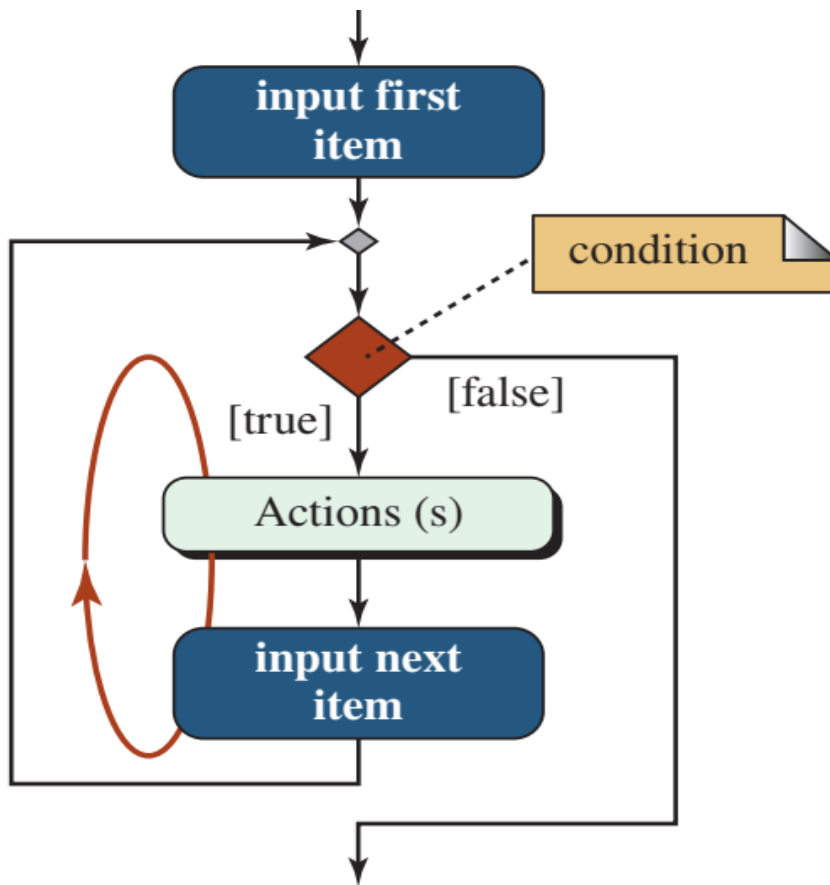
Let us assume that a professor needs to write a program to determine the average score for each student during exam. The professor gives four exams in each term to each student. Write a code to implement this logic.

The *while* Statement / *while* Loop (*continued...*)

2. Event-Controlled *while* Statement

Sometimes we want to repeat an activity, but we do not know how many times we need to do it. We know that we should stop when an event occurs. This may also happen in real life. For example, we want to do push-ups in the morning, but we do not know how far we can go. We think we should stop when we get tired. In other words, *tiredness* is the *event* we are looking for to stop the repetition. Figure 8 next slide shows an **event-controlled loop**.

The *while* Statement / *while* Loop (continued...)



Flow diagram

Note:

An event should occur to stop the loop.

```
input first item
while (event did not occur)
{
    Actions (s)
    input next item
}
```

Code

Figure 8: Flow diagram and the code for a event-controlled while statement

The *while* Statement / *while* Loop (*continued...*)

// Write a program which accepts numbers between 0 and 9 and double them.

```
#include <iostream>
using namespace std;
int main()
{
    int number;
    cout << "Enter a number between 0 and 9 inclusive:";
    cin >> number;
    while (number >= 0 && number <= 9)
    {
        cout << "Double " << number << " is " << number*2 << endl;
        cout << "Enter a number between 0 and 9 inclusive:";
        cin >> number;
    }
    cout << number << " is not between 0 and 9.\n";
    return 0;
}
```

Output

```
Enter a number between 0 and 9 inclusive:7
Double 7 is 14
Enter a number between 0 and 9 inclusive:12
12 is not between 0 and 9.
-----
Process exited after 12.35 seconds with return value 0
Press any key to continue . . .
```

The *while* Statement / *while* Loop (*continued...*)

3. Sentinel-Controlled *while* Statement

A sentinel is a guard who prevents unauthorized persons from passing a point. *In data processing, a sentinel is a value that is added to the list of data to show when we need to stop processing.* The type (data type) of the sentinel is same as the rest of the data. When we use the *while* loop as the **sentinel-controlled loop**, the sentinel is not to be processed.

Example

Assume we have a long list of positive numbers to add. We don't want to count the numbers before entering them into the computer. Since we already know that there are no negative numbers in the list, we can add a negative number at the end of the list to serve as the sentinel, as shown below: *“-1 is serving as a sentinel.”*

14 23 71 87 ... 66 12 -1

The *while* Statement / *while* Loop (continued...)

The sentinel-controlled loop

```
1  /*****
2  * Use of the sentinel-controlled while loop to find the sum      *
3  * of some numbers                                              *
4  *****/
5  #include <iostream>
6  using namespace std;
7
8  int main ()
9  {
10     // Declaration
11     int sum = 0;
12     int num;
```

The *while* Statement / *while* Loop (continued...)

```
13 // Loop including the first input
14 cout << "Enter an integer (-1 to stop): ";
15 cin >> num;
16 while (num != -1)
17 {
18     sum = sum + num;
19     cout << "Enter an integer (-1 to stop): ";
20     cin >> num ; // Sentinel update
21 }
22 // Outputting result
23 cout << "The sum is: " << sum;
24 return 0;
25 }
```

Run:

```
Enter an integer (-1 to stop): 25
Enter an integer (-1 to stop): 22
Enter an integer (-1 to stop): 12
Enter an integer (-1 to stop): 67
Enter an integer (-1 to stop): -1
The sum is: 126
```


The *for* Statement

- As we discussed previously, we can always use the *while* loop as a counter-controlled loop.
- C++ provides another loop construct, however, called **the *for* statement**, that is particularly useful when we need counter-controlled iteration.
- It combines the three elements of a loop in the loop itself:
 - i. Initialization
 - ii. Conditional test
 - iii. update
- A quick comparison between the two loops is shown in figure 10 next slide; it shows that **the *for* statement is much more compact than using a *while* statement for counter-controlled looping.**

The *for* Statement (continued...)

```
initialization;  
  
while (condition)  
{  
  
    actions  
  
    update;  
}
```

The *while* statement

```
for (initilization; condition; update)  
{  
  
    actions  
  
}
```

The *for* statement

Figure 10: Comparison between a ***while*** loop and a ***for*** loop

The *for* Statement (continued...)

i. Initialization

// Counter declaration before the header

```
int i;  
for (i = 0 ; ... ; ... )  
{  
    actions  
}
```

// Counter declaration inside the header

```
for (int i = 0 ; ... ; ... )  
{  
    actions  
}
```

In the first method, the scope of the counter extends beyond the boundary of the loop; in the second method, the scope of the counter is only inside the loop.

The *for* Statement (continued...)

ii. Condition

The condition section is the second part in the header. As in the *while* statement, *the condition in the for statement is a Boolean expression. If the condition is true, the body of the loop is executed; otherwise, the loop exits.* If the condition section is empty (between the first and the second semicolons). It is defaulted to true.

iii. Updating

The updating section is executed after the body of the loop is executed. After updating, the condition is tested again to see if the body should be executed once more.

Body- The body of the *for* statement should be the same as the body of the corresponding *while* statement with one exception; the updating of the counter is missing because it has been moved to the header.

The *for* Statement (continued...)

```
1  /*****
2  * Use of a for loop to print n integers *
3  *****/
4  #include <iostream>
5  using namespace std;
6
7  int main ()
8  {
9      // Declaration
10     int n;
11     // Get the value of n
12     cout << "Enter the number of integers to print: ";
13     cin >> n;
14     // Loop
15     for (int counter = 0; counter < n; counter++)
16     {
17         cout << counter << " ";
18     }
19     return 0;
20 }
```

The *for* Statement (continued...)

Run:

Enter the number of integers to print: 5

0 1 2 3 4

Run:

Enter the number of integers to print: 3

0 1 2

Home Assignment 2

Program 2: (Use of for loop)

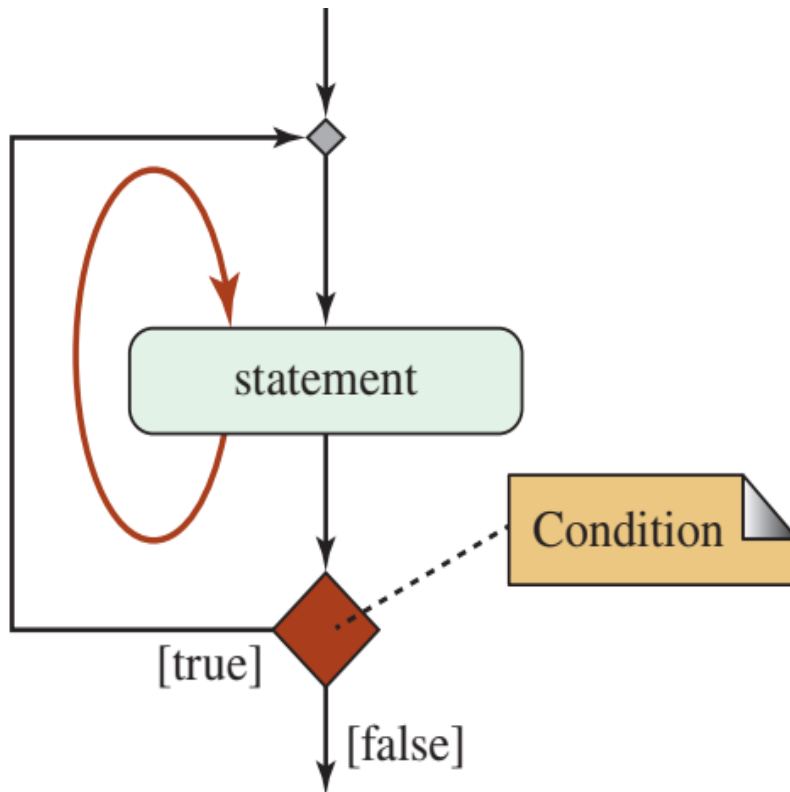
Suppose we want to print integers from 1 to 300 that are divisible by 7. The output, however, is to be printed in a table of 10 columns. Write a code to implement this logic.

Hint: Use set field function `setw( )` and it is declared inside `#include<iomanip>`.

The *do-while* Statement

- The last repetition statement we will discuss is the ***do-while statement***, in which *the logical expression is tested at the end of each iteration instead of at the beginning*. For this reason the ***do-while*** loop is a **post-test loop** as opposed to *for* loop and *while* loop which are **pre-test loops**.
- The *do-while* statement consists of five parts:
 - ❖ the reserved word *do*
 - ❖ the statement that is the body of the loop (usually a compound statement enclosed in braces)
 - ❖ the reserved word *while*
 - ❖ a logical expression in parenthesis
 - ❖ a semicolon.
- Figure 11 next slide shows the flow diagram and the syntax of the *do-while statement*.

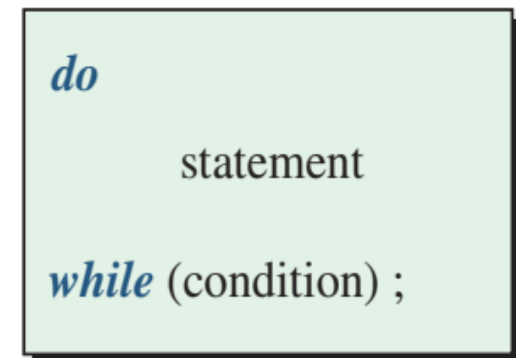
The *do-while* Statement (continued...)



Flow diagram

Note:

A do-while statement requires a semicolon.



C++ syntax

Figure 11: *Flow diagram and the code for a do-while while statement*

The *do-while* Statement (continued...)

Example: Write a C++ program to add numbers until user enters 0 using do-while loop.

```
#include <iostream>
using namespace std;
int main()
{
    float number, sum = 0.0;
    do {
        cout<<"Enter a number: ";
        cin>>number;
        sum = sum+ number;
    }
    while(number != 0.0);
    cout<<"Total sum = "<<sum;
    return 0;
}
```

Output

```
Enter a number: 6
Enter a number: 7
Enter a number: 8
Enter a number: 9
Enter a number: 0
Total sum = 30
-----
Process exited after 11.27 seconds with return value 0
Press any key to continue . . . ■
```

Nested For loops

We can have a **nested loop**, a loop inside another loop. The inside loop does not have to be of the same type as the outside loop.

Nested For loops (continued...)

Implementation of Nested Loop

```
#include <iostream>
using namespace std;
```

```
int main()
{
    int i, j;
    for (i=1; i<=4; i++) // outer loop → rows
    {
        for (j=1; j<=5; j++) // inner loop → columns
        {
            cout<<j;
        }
        cout<<endl;
    }
    return 0;
}
```

Output

```
12345
12345
12345
12345
-----
```

break and continue Statements

Recall! The **break** statement, when executed in a **switch** structure, provides an immediate exit from the switch structure. Similarly, you can use the break statement in **while**, **for**, and **do-while** loops. When the break statement executes in a repetition structure, it immediately exists from the structure. The break statement is typically used for two purposes:

- To exit early from a loop.
- To skip the remainder of the switch structure.

break and continue Statements (continued...)

How Break statement works?

```
while (test expression) {  
    statement/s  
    if (test expression) {  
        break;  
    }  
    statement/s  
}
```



The flowchart shows a loop that starts with a 'while (test expression)' condition. If the condition is true, it enters the loop body. Inside the loop, there is an 'if (test expression)' condition. If this condition is true, the flow goes to a 'break;' statement, which then exits the loop. If the 'if' condition is false, the flow goes to the 'statement/s' and then loops back to the 'while' condition.

```
do {  
    statement/s  
    if (test expression) {  
        break;  
    }  
    statement/s  
} while (test expression);
```



The flowchart shows a 'do' loop. It starts with a 'do' block containing 'statement/s' and an 'if (test expression)' condition. If the condition is true, it goes to a 'break;' statement, which then exits the loop. If the condition is false, it goes to the 'statement/s' and then loops back to the 'do' block. The loop ends with a 'while (test expression);' condition.

```
for (initial expression; test expression; update expression) {  
    statement/s  
    if (test expression) {  
        break;  
    }  
    statements/  
}
```



The flowchart shows a 'for' loop. It starts with a 'for' block containing 'initial expression', 'test expression', and 'update expression'. It then enters the loop body. Inside the loop, there is an 'if (test expression)' condition. If this condition is true, the flow goes to a 'break;' statement, which then exits the loop. If the condition is false, the flow goes to the 'statements/' and then loops back to the 'for' block. The loop ends with a closing brace '}'.

break and continue Statements (continued...)

```
#include <iostream>
using namespace std;           // A program to demonstrate working of a break statement.
int main() {
    float number, sum = 0.0;
    // test expression is always true
    while (true)
    {
        cout << "Enter a number: ";
        cin >> number;
        if (number != 0.0)
        {
            sum = sum + number;
        }
        else
        {
            // terminates the loop if number equals 0.0
            break;
        }
    }
    cout << "Sum = " << sum;
    return 0;
}
```

Output

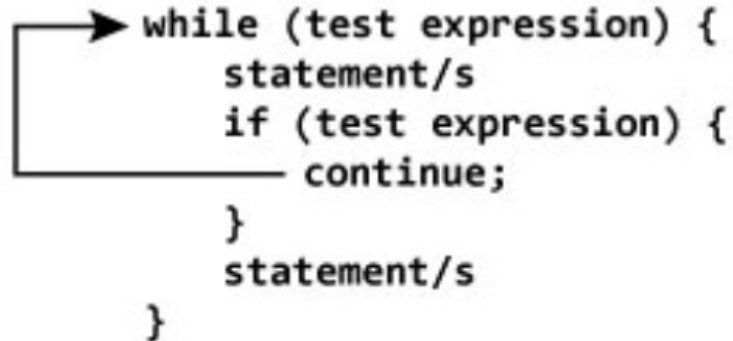
```
Enter a number: 5
Enter a number: 5
Enter a number: 5
Enter a number: 5
Enter a number: 5
Enter a number: 0
Sum = 25
-----
Process exited after 5.609 seconds with return value 0
Press any key to continue . . .
```

break and continue Statements (continued...)

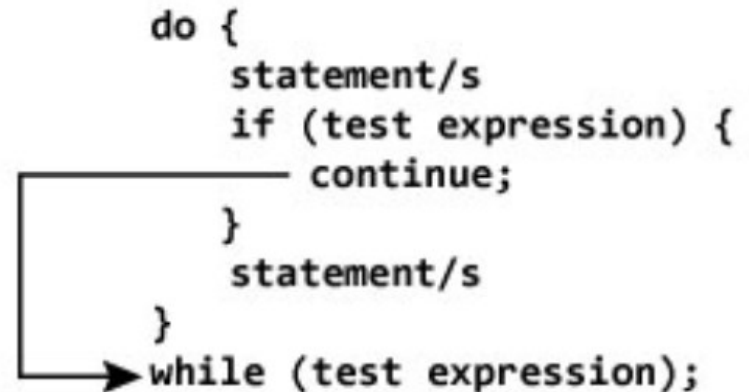
It is sometimes necessary to skip a certain test condition within a loop. In such case, `continue` statement is used in C++ programming.

break and continue Statements (continued...)

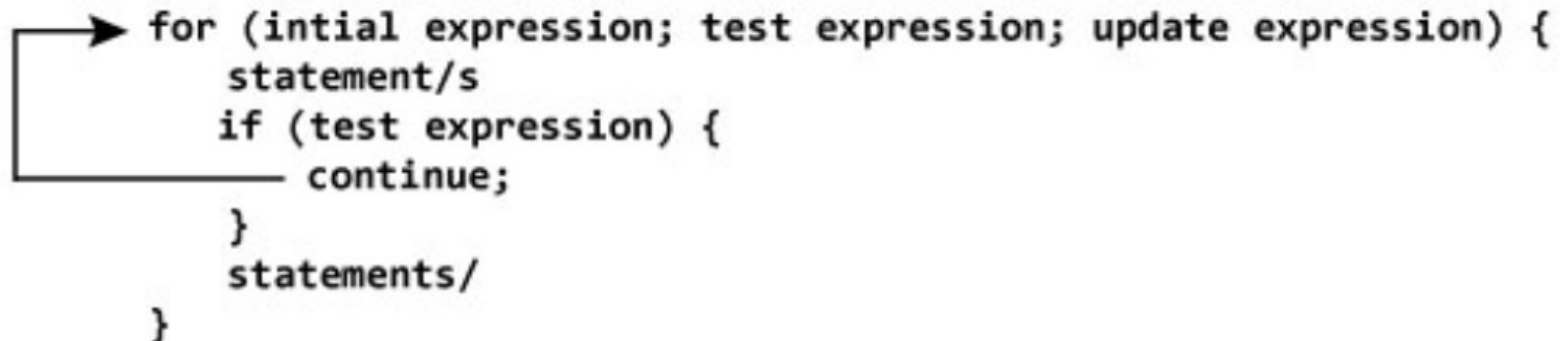
Working of continue Statement.



```
while (test expression) {  
    statement/s  
    if (test expression) {  
        continue;  
    }  
    statement/s  
}
```



```
do {  
    statement/s  
    if (test expression) {  
        continue;  
    }  
    statement/s  
}  
while (test expression);
```



```
for (initial expression; test expression; update expression) {  
    statement/s  
    if (test expression) {  
        continue;  
    }  
    statements/  
}
```

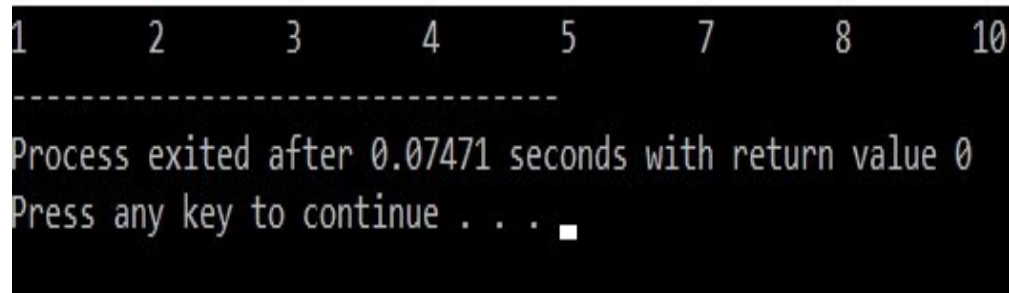
break and continue Statements (continued...)

C++ program to display integer from 1 to 10 except 6 and 9.

```
#include <iostream>
using namespace std;

int main()
{
    for (int i = 1; i <= 10; ++i)
    {
        if ( i == 6 || i == 9)
        {
            continue;
        }
        cout << i << "\t";
    }
    return 0;
}
```

Output



The screenshot shows the output of the C++ program. At the top, the integers 1 through 10 are displayed in a row, separated by spaces. Below this, a dashed line is shown. The text "Process exited after 0.07471 seconds with return value 0" is displayed, followed by "Press any key to continue . . .".

References: Textbooks

1. *“C++ Programming: From Problem Analysis to Program Design”, 6/ed by D.S. Malik, (2013), CENGAGE Learning.*
2. *“Object Oriented Programming in C++,” 4/ed by Robert Lafore (2001) .*
3. *“C++ How to Program,” 5/ed by Deitel & Deitel (2005).*
4. *“Program with C++ Object Oriented Programming Aikman Series”, by C.M Aslam and T.A Qureshi.*
5. *Related documents from open source, mainly internet.*